

Summary of Backpropagation-Free Training

Geng Tian

May 11, 2025

1 BP-Free Training of VGG16

I implemented the VGG16 by referring to the open-source code provided in [1]. The decision to use VGG16 instead of ResNet18 was based on previous experimental results: VGG8 consistently outperformed ResNet5, and VGG16 has a simpler structure compared to ResNet18. However, according to the settings used in earlier implementations, training VGG16 using the BP-free method requires an excessively large amount of memory, exceeding the total capacity available on the GPUs I have access to. To address this issue, I began exploring to trade computation time for reduced memory usage. As a result, the current training process is still ongoing and I have not yet obtained the training results.

2 Updating the Loss Calculation Method

I modified the loss computation by applying Equation 1, where each data sample is processed N times in a single epoch. For each $i \in N$, loss_i represents the standard loss during the i -th forward pass.

$$\hat{\text{loss}}_i = \text{loss}_i - \sum_{i=1}^N \frac{\text{loss}_i}{N} \quad (1)$$

Despite this change, no improvement in model accuracy was observed. The comparison between the old and new loss calculation methods is summarized in Table 1.

Table 1: Model accuracy under different loss computation strategies.

Loss Computation Strategy	Model	Accuracy (%)
Standard Loss	ResNet-5	58.48
	VGG-8	73.17
Multi-pass Mean-Centered Loss	ResNet-5	58.26
	VGG-8	72.69

3 Model Accuracy: Add Noise to the Activation Value

Table 2: Model accuracy with different noise addition method.

Loss Computation Strategy	Model	Accuracy (%)
Published results	ResNet-5	64.8
	VGG-8	77.3
Experimental results	ResNet-5	76.85
	VGG-8	71.63

In the open-source implementation of [1], both the VGG-8 and ResNet-5 models add noise to the model parameters in the first layer. However, this approach is incompatible with our proposed model splitting architecture. To address this issue, we modified the first layer by adding noise to the activation values instead of the parameters. Table 2 compares the accuracy of our modified models with the results reported in [1].

4 Model Splitting

Initially, we considered applying the BP-Free framework in split learning to reduce communication overhead. In theory, this method could halve the amount of data transmitted between the client and server since only activation values and loss values need to be exchanged during training. However, I found that this approach does not achieve the expected benefits due to the way noise gradients are computed and updated.

Specifically, in the BP-Free method, the server generates noise and updates its associated parameters using gradient information. As a result, the client must send back the gradients of these noise parameters to the server during each update. This requirement leads to a communication cost that is essentially equivalent to that of traditional backpropagation. Additionally, the engineering complexity of implementing this mechanism is significantly higher than anticipated.

To simplify the implementation while maintaining the same communication efficiency as standard backpropagation, I chose an alternative approach. In this setup, the server maintains the complete model, while the client holds only the first layer. During training, the client performs forward propagation on its local data and sends the resulting activation values to the server. Upon receiving these activations, the server creates many copies and uses them as input to its own model for further computation. After updating the server-side model, the first-layer parameters are sent back to the client for updating client’s model. This strategy ensures that the model accuracy remains unaffected while significantly reducing the implementation complexity.

References

- [1] J. Jiang, Z. Zhang, C. Xu, Z. Yu, and Y. Peng, “One forward is enough for neural network training via likelihood ratio method,” *arXiv preprint arXiv:2305.08960*, 2023.